

Experiment 1

<https://github.com/PRASHIRAWAL/R-Programming-Experiment-1>

Task 1: Import and Inspect the Dataset

Display the first six records

```
> head(air_data)
  No year month day hour PM2.5 PM10 SO2 NO2 CO O3 TEMP PRES DEWP RAIN wd WSPM
1 1 2013     3   1   0     4     4   4   7 300 77 -0.7 1023.0 -18.8  0 NNW  4.4
2 2 2013     3   1   1     8     8   4   7 300 77 -1.1 1023.2 -18.2  0  N  4.7
3 3 2013     3   1   2     7     7   5  10 300 73 -1.1 1023.5 -18.2  0 NNW  5.6
4 4 2013     3   1   3     6     6  11  11 300 72 -1.4 1024.5 -19.4  0  NW  3.1
5 5 2013     3   1   4     3     3  12  12 300 72 -2.0 1025.2 -19.5  0  N  2.0
6 6 2013     3   1   5     5     5  18  18 400 66 -2.2 1025.6 -19.6  0  N  3.7
  station
1 Aotizhongxin
2 Aotizhongxin
3 Aotizhongxin
4 Aotizhongxin
5 Aotizhongxin
6 Aotizhongxin
```

Display the structure of the dataset

```
> str(air_data)
'data.frame':   35064 obs. of  18 variables:
 $ No      : int   1  2  3  4  5  6  7  8  9 10 ...
 $ year    : int  2013 2013 2013 2013 2013 2013 2013 2013 2013 2013 ...
 $ month   : int   3  3  3  3  3  3  3  3  3  3 ...
 $ day     : int   1  1  1  1  1  1  1  1  1  1 ...
 $ hour    : int   0  1  2  3  4  5  6  7  8  9 ...
 $ PM2.5   : num   4  8  7  6  3  5  3  3  3  3 ...
 $ PM10    : num   4  8  7  6  3  5  3  6  6  8 ...
 $ SO2     : num   4  4  5 11 12 18 18 19 16 12 ...
 $ NO2     : num   7  7 10 11 12 18 32 41 43 28 ...
 $ CO      : int  300 300 300 300 300 400 500 500 500 400 ...
 $ O3      : num  77 77 73 72 72 66 50 43 45 59 ...
 $ TEMP    : num  -0.7 -1.1 -1.1 -1.4 -2 -2.2 -2.6 -1.6 0.1 1.2 ...
 $ PRES    : num  1023 1023 1024 1024 1025 ...
 $ DEWP    : num  -18.8 -18.2 -18.2 -19.4 -19.5 -19.6 -19.1 -19.1 -19.2 -19.3 ...
 $ RAIN     : num   0  0  0  0  0  0  0  0  0  0 ...
 $ wd      : chr   "NNW" "N"  "NNW" "NW" ...
 $ WSPM    : num   4.4 4.7 5.6 3.1 2 3.7 2.5 3.8 4.1 2.6 ...
 $ station : chr   "Aotizhongxin" "Aotizhongxin" "Aotizhongxin" "Aotizhongxin" ...
```

Display the number of rows and columns, check whether the dataset contains missing values and display the total number of missing values.

```
> dim(air_data)
[1] 35064    18

> any(is.na(air_data))
[1] TRUE

> sum(is.na(air_data))
[1] 7271
> |
```

Task 2: Understand NA, NULL, and NaN

NA indicates a missing observation, NULL represents an empty object, and NaN indicates an undefined numerical result. The corresponding functions correctly identified each value type.

Temperature Vector:

```
> print(temperature)
[1] 28 30 NA 32

> cat("\nMissing Object:\n")
Missing Object:

> print(missing_object)
NULL

> cat("\nUndefined Value:\n")
Undefined Value:

> print(undefined_value)
[1] NaN

> cat("\nChecking NA:\n")
```

Checking NA:

```
> print(is.na(temperature))
[1] FALSE FALSE TRUE FALSE

> cat("\nChecking NULL:\n")
Checking NULL:

> print(is.null(missing_object))
[1] TRUE

> cat("\nChecking NaN:\n")
Checking NaN:

> print(is.nan(undefined_value))
[1] TRUE
```

Task 3: Create a Missing-Value Summary Function

```
> # Task 3: Missing Value Summary
>
> missing_summary <- function(data)
+ {
+   selected_variables <- c("PM2.5", "PM10", "SO2", "NO2", "TEMP", "WSPM" .... [TRUNCATED]
> missing_table <- missing_summary(air_data)
> print(missing_table)
Variable Total_Records Missing_Values Missing_Percentage
1 PM2.5 35064 925 2.64
2 PM10 35064 718 2.05
3 SO2 35064 935 2.67
4 NO2 35064 1023 2.92
5 TEMP 35064 20 0.06
6 WSPM 35064 14 0.04
7 wd 35064 81 0.23
```

Task 4: Identify Invalid Numerical Results

```
> # Task 4: Identify Invalid Numerical Results
>
> air_data$pollution_ratio <- air_data$PM2.5 / air_data$PM10
> cat("Number of NA values:",
+   sum(is.na(air_data$pollution_ratio)), "\n")
Number of NA values: 934
> cat("Number of NaN values:",
+   sum(is.nan(air_data$pollution_ratio)), "\n")
Number of NaN values: 0
> cat("Number of Infinite values:",
+   sum(is.infinite(air_data$pollution_ratio)), "\n")
Number of Infinite values: 0
> air_data$pollution_ratio[
+   is.nan(air_data$pollution_ratio) |
+   is.infinite(air_data$pollution_ratio)
+ ] <- NA
```

```
> cat("\nAfter Replacement\n")

After Replacement

> cat("NA values:",
+      sum(is.na(air_data$pollution_ratio)), "\n")
NA values: 934

> cat("NaN values:",
+      sum(is.nan(air_data$pollution_ratio)), "\n")
NaN values: 0

> cat("Infinite values:",
+      sum(is.infinite(air_data$pollution_ratio)), "\n")
Infinite values: 0
```

Task 5: Handle Missing Numerical Values Using a Loop

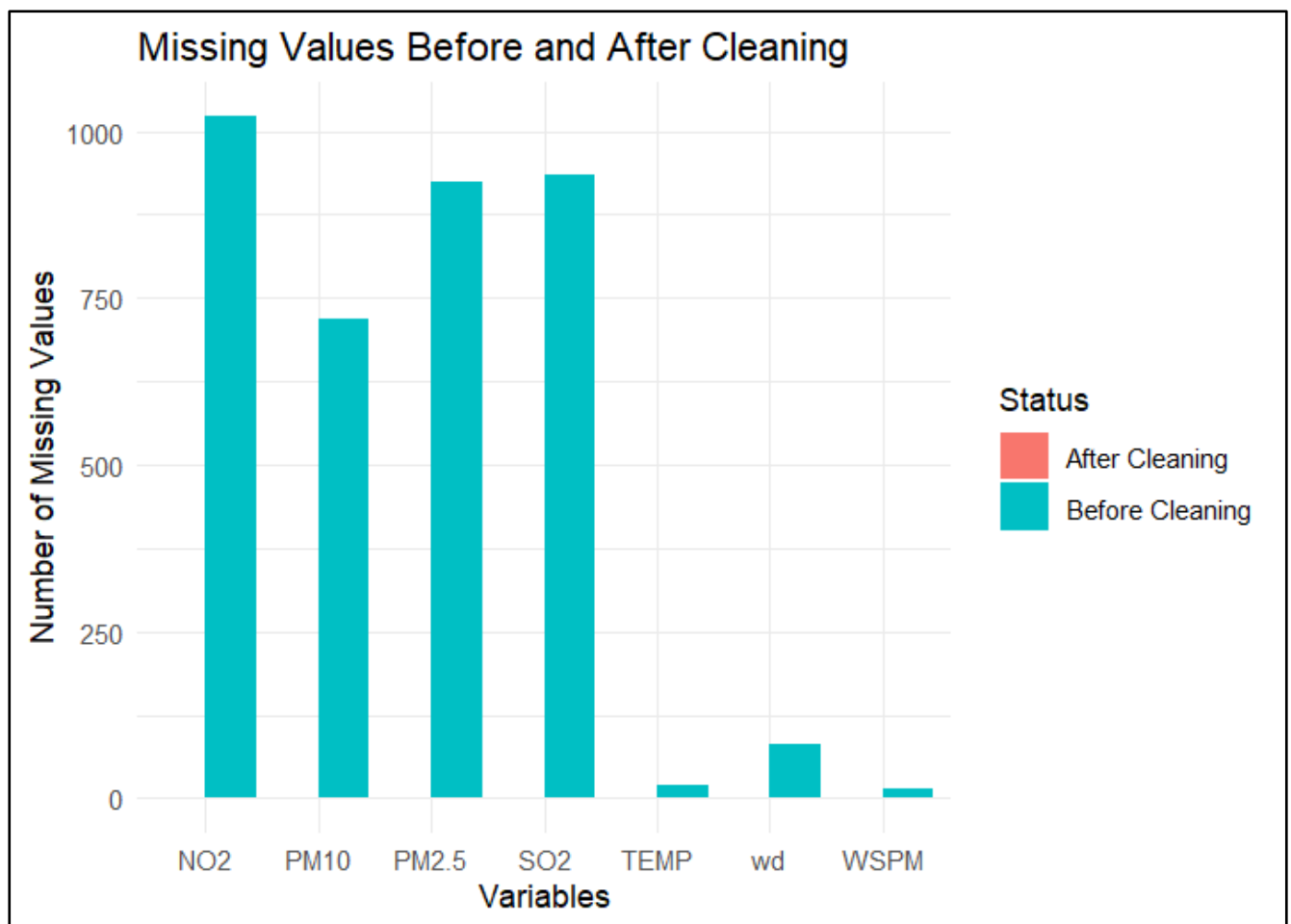
```
Variable: PM2.5
Missing Before: 925
Median Used: 58
Missing After: 0
Variable: PM10
Missing Before: 718
Median Used: 87
Missing After: 0
Variable: SO2
Missing Before: 935
Median Used: 9
Missing After: 0
Variable: NO2
Missing Before: 1023
Median Used: 53
Missing After: 0
Variable: TEMP
Missing Before: 20
Median Used: 14.5
Missing After: 0
Variable: WSPM
Missing Before: 14
Median Used: 1.4
Missing After: 0
```



Task 8 – Compare Missing Values Before and After Cleaning

```
> print(comparison_table)
Variable Missing_Before Missing_After Values_Replaced
1 PM2.5 925 0 925
2 PM10 718 0 718
3 SO2 935 0 935
4 NO2 1023 0 1023
5 TEMP 20 0 20
6 WSPM 14 0 14
7 wd 81 0 81
>
```

Task 9 – Generate One Visualization



Task 10 – Export the Cleaned Dataset

```
> # Task 10: Export the Cleaned Dataset
>
> write.csv(
+   air_data,
+   "cleaned_air_quality_data.csv",
+   row.names = FALSE
+ )

> cat("Cleaned dataset exported successfully.\n")
Cleaned dataset exported successfully.
> |
```

The Beijing Multi-Site Air Quality dataset was successfully cleaned using R programming techniques. Missing values in numerical variables were identified and replaced using the median, while missing values in the categorical variable wd were replaced using the mode. A user-defined function was developed to summarize missing values and another function with **tryCatch()** was implemented to handle errors gracefully. A comparison table and bar chart confirmed that all selected variables were free of missing values after cleaning. The cleaned dataset was then exported for future analysis. This experiment demonstrated the practical use of loops, functions, conditional statements, error handling and data preprocessing techniques in R.

This experiment provided practical experience in cleaning real-world datasets using R. It demonstrated how to identify and handle different types of missing values (NA, NULL and NaN), apply median and mode imputation, create reusable functions, use loops to avoid repetitive code and implement robust error handling using **tryCatch()**. The cleaned dataset was successfully exported and is ready for further statistical analysis or machine learning applications.